

PA1 explained from scratch

Sep 23, 2026 · @zylah

A plain-language companion for writing the PA1 report: what each task is actually doing, why, and what your own results mean.

Start here: the one idea behind all four tasks

Every machine learning result you have ever seen rests on one assumption, and this whole assignment is about what happens when it fails.

The assumption is called **IID**: independent and identically distributed. "Identically distributed" is the part that matters here. It says the images you test on come from the same source as the images you trained on — same kinds of photos, same lighting, same object categories. If that holds, test accuracy is an honest estimate of how the model will do in the world. If it does not hold, test accuracy is a number about a world the model will never see again.

The assignment title is "Learning beyond IID". Each task breaks the assumption in a different way and asks: what does the model do, how do we measure it, and can we fix it?

The three ways it breaks

Task	What changes at test time	What stays the same	The question
1	The <i>appearance</i> of the images (colour removed, shifted sideways, patches shuffled)	The classes, the dataset	What information is the model actually using — shape, texture, or position?
2	The <i>domain</i> — photos at train time, sketches at test time — but you have unlabelled sketches to look at	The 7 classes	Can unlabelled target images help you adapt?
3	The domain again, but you never see the target domain at all	The 7 classes	Can you train to be robust to a domain you cannot look at?

Task	What changes at test time	What stays the same	The question
4	New <i>classes</i> appear that were never in training	The image style (both CIFAR)	Can the model say "I don't know" instead of guessing?

Tasks 2 and 3 are the same shift with different rules about what you're allowed to peek at. Task 4 is a different kind of shift entirely: not "the same things look different" but "these are not the same things at all".

The sentence your report is arguing for

A representation that survives outside IID has to do three things at once:

- **preserve** what defines the class (Task 1: is it shape or texture? Tasks 2–3: does it survive the photo→sketch jump?)
- **suppress** what is incidental (colour, domain style, position)
- **reject** what it has no business labelling (Task 4: the unknown classes)

Your four tasks are evidence for and against each of those three verbs. That is the through-line of the discussion section, and section 8 of this doc assembles it for you.

How to use this doc

Read section 2 (Background) once, slowly. Then read the section for whichever task you are writing up. Each task section has the same shape: **what the handout asks → the concepts you need → step by step what the code does → your actual numbers → what to say about them**. The glossary at the end is for when a word appears and you have forgotten it.

Every task section ends with your actual numbers and a suggested framing for them.

Background in plain words

A classifier in four stages. Image → **backbone** (a stack of convolutions or transformer blocks that turns pixels into a list of ~512–2048 numbers) → **features** (that list, also called the *representation* or *embedding*) → **head** (one matrix multiply) → **logits** (one raw score per class) → **softmax** (turns logits into probabilities summing to 1). The predicted class is the largest logit. Everything in this assignment is about the features: the backbone is where the knowledge lives, the head is trivial.

Frozen vs fine-tuned. Frozen = the backbone's weights never change; you only train the head. That isolates *what the pretrained features already contain* — Task 1's whole point. Fine-tuned = everything updates (Tasks 2–4).

The three splits. Train (fit weights), **validation** (choose the epoch/hyperparameter), test (report once). The rule that matters: anything you *select on* is no longer a fair test. Task 4's leakage rules exist for exactly this — the unknown classes must not touch training or threshold-setting.

Checkpoint selection. You save the model at the epoch with the best validation accuracy, not the last epoch. Models get worse near the end; the validation set tells you where to stop.

Domain. A style of image, not a class. PACS has 4 domains (Photo, Art painting, Cartoon, Sketch) × the same 7 classes. A dog is a dog in all four, but the pixels look nothing alike.

Accuracy vs macro-F1. Accuracy = fraction correct; it flatters a model that ignores rare classes. **Macro-F1** computes F1 per class and averages the seven equally, so collapsing to one class scores near zero. The handout asks for macro-F1 because collapse is a real failure mode here — and in Task 3 it actually happened.

Seed. The one number (6304 everywhere) that fixes all randomness — weight init, shuffling, augmentation — so a rerun reproduces bit-for-bit. Yours did: Task 4 run 2 matched run 1's checkpoint hashes exactly.

Epoch, batch, learning rate. One epoch = one pass over the training set. A batch = the ~128 images processed at once. The learning rate is the step size; "cosine decay" means it shrinks smoothly to zero over the run.

BatchNorm (needed for Tasks 2 and 4). A layer that normalises each channel by the mean and variance of the current batch. In training it uses the batch's own statistics; at test time it uses **running statistics** accumulated during training. If the batches it saw during training were not representative of test images, those stored numbers are wrong and accuracy drops even though the weights are fine. That single fact explains the Task 2 frozen-BN policy and the Task 4 PROSER bug.

Task 1 — Inductive biases and representations

Numbers below are from your `task1.ipynb` run (STL-10, seed 6304). Four evaluated models: linear heads on three frozen backbones, plus CLIP zero-shot.

The question. Three pretrained backbones all score well on ImageNet. Do they get there the same way? An **inductive bias** is the built-in preference a model has for one kind of explanation over another — e.g. deciding "cat" from its outline vs from its fur texture. Same accuracy, different bias, very different behaviour off-distribution.

The setup. Three frozen backbones — ResNet-50 (`IMAGENET1K_V2`), ViT-B/16, OpenCLIP ViT-B-32 (`openai`) — each with a linear head trained on STL-10. Frozen is essential: you are probing the *pretrained* features, not what a head can learn to fix.

The six probes

1. **Clean baseline.** Accuracy with no intervention. Everything later is read as a drop from this.
2. **Colour interventions.** Grayscale, plus one more (e.g. channel shuffle or colour jitter). If accuracy barely moves, colour was not load-bearing.
3. **Cue conflict (AdaIN).** The core experiment. AdaIN (adaptive instance normalisation) repaints image A in the *texture statistics* of image B, so you get a dog-shaped thing with elephant texture. ≥ 5 class pairs, ≥ 200 conflict images. Each prediction is scored: matches the shape source → **shape vote**; matches the texture source → **texture vote**; anything else → neither.
 - **Shape bias** = $N_{\text{shape}} / (N_{\text{shape}} + N_{\text{texture}})$. 1.0 = pure shape, 0.0 = pure texture, 0.5 = balanced.
 - **Coverage** = $(N_{\text{shape}} + N_{\text{texture}}) / N_{\text{total}}$ — the fraction of conflicts where the model picked *one of the two* sources at all. Low coverage means the shape-bias number is computed on a small, possibly unrepresentative slice, so always report it alongside.
4. **Translation.** Shift the image 0 / 8 / 16 / 32 px with reflection padding. CNNs are built to be shift-tolerant; ViTs are not, but learn it. Watch how fast each curve falls.
5. **Patch shuffle (4×4).** Cut into 16 tiles and permute them. This destroys global shape while leaving local texture intact. A texture-driven model barely notices; a shape-driven one collapses. It is the cue-conflict result seen from the other side.
6. **Representation stability.** Two numbers per intervention: **prediction consistency** = fraction of images whose predicted label is unchanged; **cosine stability** = average cosine similarity between the feature vector before and after. These come apart — features can drift a lot while the argmax survives — and that gap is worth a sentence.

Plus a t-SNE or UMAP plot: a 2-D picture of the feature space, useful for showing whether classes stay separated under the intervention. Treat it as illustration; the distances in it are not quantitative.

Your results

Splits: 4,000 train / 1,000 val / 500 balanced test (50 per class). Cue conflicts: 240 generated across 10 directed class pairs, 231 accepted after your manual review, then balanced to 22 per cell = **220 evaluated**.

Model	Clean	Grayscale Δ	Colour Δ	Shuffle Δ	Shuffle consistency	Shape bias	Coverage
ResNet-50	96.6	-2.2	-3.0	-10.2	88.8	49.6%	50.5%
ViT-B/16	96.8	-2.4	-3.6	-6.0	92.8	76.9%	59.1%
CLIP (linear head)	97.4	-3.4	-4.6	-11.6	86.2	80.3%	60.0%
CLIP (zero-shot)	96.4	-3.2	-4.4	-13.8	83.6	85.4%	62.3%

All four are within 1.0 point clean (96.4–97.4) and behave completely differently off-distribution. That single sentence justifies the whole task — closed-set accuracy does not identify the model.

Shape bias orders cleanly: ResNet-50 (49.6%) $\ll$ ViT-B/16 (76.9%) < CLIP head (80.3%) < CLIP zero-shot (85.4%). The convolutional backbone is balanced between the two cues; both transformers and especially the language-supervised one favour shape. Report **coverage alongside** — at 50–62%, roughly 40% of conflicts got a third class entirely, so shape bias is computed on a decided minority and is noisier than the decimals suggest. Coverage rising with shape bias is itself informative: the more shape-biased models are also the more decisive ones on conflicts.

The dissociation worth a paragraph: shape bias and patch-shuffle robustness are not the same axis. If shape bias simply meant “uses global form”, the most shape-biased model should suffer most from destroying global form — and CLIP zero-shot does (–13.8, the largest drop, consistency 83.6). But ViT-B/16 is *both* strongly shape-biased (76.9%) *and* the most shuffle-robust (–6.0). The explanation is architectural: ViT tokenises the image into patches and adds position embeddings, so permuting 4×4 tiles leaves the token contents intact and only disturbs position — a much smaller perturbation than for a

convolution stack whose receptive fields straddle tile boundaries. So: *cue preference and spatial-structure dependence are separable properties, and patch shuffle measures the second, not the first.*

Colour is secondary everywhere — 2.2 to 4.6 points — but consistently costs CLIP more than ResNet-50, which fits CLIP's text-aligned training where colour words carry real semantic weight.

Translation: everyone is robust, and the textbook prediction fails.

Displacement	0 px	8 px	16 px	32 px
ResNet-50	96.60	96.00	96.10	95.45
ViT-B/16	96.80	96.70	96.70	96.35
CLIP head	97.40	96.40	96.25	95.80
CLIP zero-shot	96.40	96.30	95.85	94.90

The expectation is that the CNN's built-in shift-equivariance beats the ViT. It does not: ViT loses 0.45 points at 32 px, ResNet-50 loses 1.15, CLIP zero-shot 1.50. Two honest caveats to state — STL-10 objects are near-centred and the images are upsampled to 224, so 32 px is a modest fraction of the frame; and reflection padding preserves edge statistics rather than introducing black borders. The defensible claim is *translation up to 32 px is not a discriminating probe on this dataset*, which is a legitimate finding about the probe.

Representation stability — read it normalised

Raw cosine similarity is **not comparable across backbones**, because their feature spaces have different natural angles. The random-pair baseline (cosine between two unrelated images) is 0.226 for ResNet-50, 0.104 for ViT-B/16 and **0.612 for CLIP** — CLIP's features live in a narrow cone, so 0.65 there means something very different from 0.65 elsewhere. Normalise as $(\cos - \text{baseline}) / (1 - \text{baseline})$:

Intervention	ResNet-50	ViT-B/16	CLIP
Grayscale	0.783 → 0.72	0.738 → 0.71	0.898 → 0.74
Shuffle	0.710 → 0.63	0.664 → 0.63	0.727 → 0.30
Translation 32 px	0.931 → 0.91	0.942 → 0.94	0.970 → 0.92

Intervention	ResNet-50	ViT-B/16	CLIP
Cue conflict	0.259 → 0.04	0.243 → 0.16	0.655 → 0.11

Three readings. First, **colour removal is nearly identical across backbones once normalised** (0.71–0.74) — the raw numbers hid that completely. Second, **CLIP's representation is by far the most disturbed by patch shuffle** (0.30 vs 0.63), which the raw cosine reversed. Third, **AdaIN cue conflict essentially relocates the representation** for all three (0.04–0.16): restyling does not nudge the feature, it moves it somewhere close to unrelated. That is the strongest single stability result in the task, and it is only visible after normalisation — worth saying you did it.

Features and predictions come apart, measurably. On cue conflicts, images whose prediction survived kept much higher feature similarity than those whose prediction flipped: ResNet-50 0.425 vs 0.201 (57 unchanged, 163 changed), ViT 0.381 vs 0.128 (100 vs 120), CLIP 0.709 vs 0.606 (104 vs 116). The feature drift *predicts* the label flip rather than being independent of it, which is the cleanest link you have between the representation-level and prediction-level metrics.

The zero-shot vs linear-head agreement probe. Same CLIP features, two different readouts: they agree 96.6% clean, 95.8% grayscale, 94.0% colour, 87.4% shuffle, and only **63.2% on cue conflicts**. Under cue conflict the features are ambiguous enough that the *decision rule* — not the representation — determines the answer. This is a nice argument that shape bias is a property of the (backbone, head) pair, not of the backbone alone, and it explains why CLIP zero-shot's shape bias (85.4%) exceeds CLIP head's (80.3%) on identical features.

What to write. The two tables above plus the normalised stability table, then: (1) equal clean accuracy, divergent biases; (2) the shape-bias ordering with coverage caveat; (3) the shape-bias / shuffle dissociation; (4) cue conflict relocates representations, and the drift predicts the flips; (5) the readout matters as much as the backbone. Use the t-SNE figures as illustration for (4) only — do not quote distances off them.

Task 2 — Unsupervised domain adaptation on PACS

The question. Train on Photo + Art + Cartoon (labelled), test on Sketch. You are allowed to *look at* Sketch images during training but never at their labels — that is what "unsupervised" means here. Does using them help?

The shared idea. All three methods assume: if the features of source and target images become statistically indistinguishable, the classifier trained on source features will work on target features. So each adds a second loss that pushes the two feature distributions together, alongside the ordinary classification loss.

The four methods

- **Source-only (ERM).** No target images at all. Plain cross-entropy on the three source domains. This is the baseline everything is compared against. ERM = empirical risk minimisation, the formal name for "minimise average training loss".
- **DAN — aligning by distance (MMD).** **MMD** (maximum mean discrepancy) is a single number measuring how far apart two clouds of points are: map both through a kernel, compare their means. Here, 3 RBF kernels at $0.5\times$, $1\times$ and $2\times$ the median pairwise squared distance (the median heuristic — it sets the kernel width to the data's own scale). Total loss = classification + $\lambda \cdot \text{MMD}$, $\lambda = 1$.
- **DANN — aligning by adversary.** Add a second head, a **domain discriminator**, trained to guess "source or target?" from the features. The backbone is trained to *fool* it. The trick is the **gradient reversal layer**: forward pass does nothing, backward pass multiplies the gradient by $-\alpha$, so one optimiser trains both directions at once. α ramps up as $\alpha(p) = 2/(1+\exp(-10p)) - 1$ over training, so alignment pressure starts at zero and grows — otherwise the adversary wrecks the features before the classifier has learned anything.
- **CDAN — conditional alignment.** DANN's flaw: it aligns the clouds as *wholes*, so it can happily map source-dogs onto target-horses. CDAN feeds the discriminator the outer product $g(x) = \text{vec}(f \otimes p)$ of the feature f and the class probabilities p , so alignment must hold class-by-class.

Domain separability probe. A diagnostic, not a method: fit a logistic regression ($C = 1$, 70/30 split) to predict the domain from the frozen features. 100% = domains are trivially separable; 50% = chance, i.e. perfectly aligned. It tells you whether the alignment loss *did its job*, which is a separate question from whether that helped accuracy.

Frozen BatchNorm. The running statistics stay at their pretrained values. Otherwise the target images (a quarter of every batch) would silently shift the normalisation statistics, and you would not know whether an improvement came from your clever loss or from that accidental recalibration.

Protocol. AdamW at $1e-4$, ≤ 30 epochs, early stopping with patience 5, batches of 8 per source domain + 24 target images.

Your results

Method	Source F1 (mean)	Sketch acc	Sketch F1	Δ vs source-only	Domain separability	Best epoch
Source-only	94.14	69.59	72.41	—	99.86%	15
DAN ($\lambda=1$)	95.02	58.59	55.16	-11.00	82.28%	10
DANN ($\alpha=1$)	35.04	26.44	18.91	-43.14	98.08%	18
CDAN	88.75	58.49	58.74	-11.10	99.45%	1

The finding: every adaptation method made it worse. This is a real result, not a bug, and it is the most interesting thing in the task — write it up confidently.

The diagnostics say *why*, and they say something different for each method:

- **DAN** genuinely aligned (separability 99.86 $\rightarrow$ 82.28) and still lost 11 points. So alignment was achieved and was not the thing that helps — it removed domain information that the classifier was using.
- **DANN at $\alpha=1$** destroyed the features outright: source F1 fell to 35.04, meaning it can no longer classify the data it has *labels* for. Separability stayed at 98.08%, so it did not even align. The adversary won and the representation collapsed.
- **CDAN** stopped at epoch 1 — early stopping selected the model before adaptation had done anything, and separability stayed at 99.45%. Its -11 points come from an under-trained model, not from alignment.

The controlled studies

DAN λ (alignment strength):

λ	Source F1	Sketch acc	Separability
0.1	93.90	67.01	97.25%
1	95.02	58.59	82.28%
10	5.07	4.07	— (collapse)

A clean monotone story: more alignment $\rightarrow$ lower separability $\rightarrow$ worse target accuracy, ending in total collapse at $\lambda=10$. Even the gentlest setting ($\lambda=0.1$, 67.01) does not beat doing nothing (69.59).

DANN α (adversary strength): $\alpha=0.25 \rightarrow 88.75$ source / 52.30 Sketch; $\alpha=0.5 \rightarrow 44.74$ / 15.47; $\alpha=1 \rightarrow 35.04$ / 26.44. Same shape: the adversary trades away the representation faster than it buys alignment.

How to frame it in the report. Do not write "adaptation failed". Write: *on PACS $\rightarrow$ Sketch with a frozen-BN, fixed-budget protocol, marginal feature alignment is achievable (DAN) but is not sufficient for transfer, and the strength of alignment trades against source accuracy monotonically.* That is a claim your λ and α sweeps actually support.

Task 3 — Domain generalization

The question. Same PACS setup, one rule removed: you may not look at Sketch at all, not even unlabelled. You have three source domains and must train something that survives a fourth you have never seen. This is the harder and more realistic setting — in deployment you rarely have target images in advance.

The logic behind every DG method. With no target data, the only signal available is the *variation among your sources*. If a feature is stable across Photo, Art and Cartoon, it is more likely to be about the object than about the style. So: either force the source domains to align with each other, or find a solution that is insensitive to perturbation.

The three methods

- **ERM.** Literally the Task 2 Source-only checkpoint, reused and never retrained — so the comparison is exact, with no rerun noise.
- **DAN-DG.** DAN's MMD loss, but applied **pairwise between the three source domains** instead of source-vs-target. $\lambda_{\text{DG}} = 1$ by default. "Make my sources look alike, and hope the fourth comes along for free."
- **SAM (sharpness-aware minimisation).** Not an alignment method at all. The idea: a loss minimum sitting in a wide flat basin generalises better than one at the bottom of a narrow spike, because a small shift in the data distribution moves you a little way along the surface and a flat basin forgives that. SAM implements it with two passes per batch: first compute the gradient and step *uphill* by $\rho = 0.05$ to find the worst nearby point, then compute the gradient *there* and apply it back at the original weights. Non-adaptive ρ , so the radius is the same for every parameter. Cost: double the compute.

The two diagnostics

- **Source separability.** The same logistic-regression probe, now predicting which of the *three source* domains a feature came from. Chance is 33.3%. Low = the sources are aligned.
- **Sharpness proxy.** One normalised gradient-ascent step of radius 0.05 on a fixed validation batch (32 per source); report the loss increase. Big increase = sharp minimum, small = flat. It directly tests SAM's premise.

Your results

Method	Source F1 (mean)	Worst-domain F1	Sketch acc	Sketch F1	Source separability	Sharpness Δ loss
ERM	94.14	90.74	69.59	72.41	87.71%	+0.286
SAM ($\rho=0.05$)	95.08	90.98	68.08	72.25	87.38%	+0.110
DAN-DG $\lambda=0.1$	95.26	93.32	68.16	71.12	79.07%	+0.174
DAN-DG $\lambda=1$	5.07	4.21	4.07	1.12	48.5%	+0.040
DAN-DG $\lambda=10$	— collapse	—	—	—	45.85%	+0.007

Finding 1: nothing beat ERM, and the gaps are noise. McNemar's test on Sketch: SAM vs ERM -1.50 points, $p_{\text{holm}} = 0.132$; DAN-DG $\lambda=0.1$ vs ERM -1.43, $p_{\text{holm}} = 0.196$; $\lambda=0.1$ vs SAM +0.08, not significant. So the honest claim is **no method is distinguishable from ERM**, not "ERM won". That matters — a 1.5-point gap you cannot separate from noise is not a result about methods.

Finding 2: SAM did exactly what it promises — and it did not help. Sharpness fell from +0.286 to +0.110, a 62% reduction. The mechanism worked; flatness simply did not buy target accuracy here. This is your cleanest "mechanism verified, benefit absent" result, and it is worth a paragraph because it is only sayable *because* you measured the mechanism separately from the outcome.

Finding 3: the $\lambda=1$ collapse, fully diagnosed. DAN-DG at $\lambda=1$ does not merely underperform, it degenerates:

- predicts "**person**" for **100%** of Sketch images
- a class probe on its features gets **28.5%** (ERM: 93.8%; chance: 14.3%) — the class information is nearly gone from the representation, not just from the head

- mean feature norm **1.774 vs ERM's 69.650** — the features have been crushed towards zero
- biased MMD **0.483** against an estimator floor of **0.450**

That last line is the explanation. The **biased** MMD estimator keeps the self-comparison terms, which gives it a non-zero floor that shrinking the features approaches. So the optimiser found a shortcut: crush every feature towards zero, drive MMD down to its floor, ignore the classification loss. Separability agrees — 48.5% at $\lambda=1$ and 45.85% at $\lambda=10$, against 87.71% for ERM and 33.3% chance. The domains are no longer separable, but only because nothing is: the probe cannot read domain out of features that carry no information at all.

The unbiased-estimator diagnostic. Rerunning with the **unbiased** MMD estimator (self-terms removed; it can go negative, so there is no floor to exploit) removes the collapse entirely: source F1 93.72, separability 71.43%, Sketch 67.09 (F1 70.67). Still -2.49 points vs ERM, and this time it *is* significant ($p_{\text{holm}} = 0.009$). Read: the estimator choice was the cause of the collapse, and once fixed, source-domain MMD alignment reliably *hurts* rather than catastrophically failing. This is a genuinely good finding — most people would have reported the collapse as "DAN-DG is unstable" and stopped.

Finding 4: per-class, it is a trade not a gain. On Sketch, ERM: dog 53.2, elephant 89.3, giraffe 45.3, guitar 83.4, horse 77.5, house 87.5, person 70.0. SAM moves horse to 56.5 (-171 images). DAN-DG $\lambda=0.1$ lifts giraffe to 70.0 (+186 images) but drops horse to 53.3 (-197). The aggregate numbers hide a large reshuffle underneath — alignment reallocates which classes work, it does not add accuracy.

Cross-task check. Task 2's DAN $\lambda=0.1$ vs Task 3's DAN-DG $\lambda=0.1$: -1.15 points, $p_{\text{holm}} = 0.31$, not significant. So *seeing unlabelled target data bought nothing measurable over aligning sources alone* — a direct, quotable answer to the question "was the extra access in Task 2 worth anything?"

Task 4 — Open-set recognition

The question. Tasks 1-3 change how things look. Task 4 introduces things that were never in the training set at all. A normal classifier *cannot* say "none of these" — softmax sums to 1, so it must pick one of the ten. Open-set recognition adds the missing option.

The setup. Knowns = the 10 CIFAR-10 classes. Unknowns = 16 fixed CIFAR-100 test classes, 800 images each: **8 near** (bus, pickup_truck, motorcycle, tractor, wolf, fox, leopard, camel — things resembling a CIFAR-10 class) and **8 far** (bottle, bowl, chair, clock,

keyboard, mushroom, sunflower, wardrobe). That split is the experiment: rejecting a wardrobe should be easy, rejecting a wolf hard. Backbone: CIFAR ResNet-18 (3×3 stride-1 first conv, no maxpool — the ImageNet stem throws away too much of a 32×32 image).

Leakage. No unknown image may touch training, checkpoint selection or the threshold. Your repo enforces this with a SHA-256 **freeze record** over 12 code files plus a logged timestamp of first CIFAR-100 access — 07:35:54 freeze, 08:27:39 first access. That ordering is the proof, and it belongs in the report.

Two scores per image

Every method produces (a) a class prediction and (b) an **unknownness score**. You then compare that score to a threshold τ and either accept the prediction or reject the image. Four post-hoc scores, all computed from the *same* trained Vanilla model — no retraining, which is why they are called post-hoc:

- **MSP** — maximum softmax probability. The naive baseline. Its weakness: softmax normalises away the overall magnitude.
- **MLS** — maximum logit score, i.e. before softmax. Keeps the magnitude MSP discards.
- **Energy** — $-\log\text{sumexp}$ of the logits. A smoothed version of MLS that accounts for all ten rather than just the top one.
- **Mahalanobis** — ignores logits. Fit a Gaussian per class in feature space with a shared covariance; score = distance to the nearest class centre. It asks "does this live where training data lived?" rather than "how confident is the head?".

Two trained methods

- **GCSC** (good closed-set classifier). The claim from the literature: you do not need a special OSR method — just train a *better* closed-set classifier and rejection improves for free. Implemented here as stronger augmentation, RandAugment(2, 9). Your job is to test whether closed-set gains carry over to open-set.
- **PROSER** (placeholder classifiers). Adds **5 dummy logits** that represent "other". They are trained to sit *between* the known classes: manifold mixup after layer2 blends two training images ($\lambda \sim \text{Beta}(2,2)$) and the dummies are trained to win on the blend, which is a stand-in for "an image that is not cleanly any known class". $\beta = 1$, $\gamma = 0.1$; detection score read at temperature $T = 1024$.

Threshold and metrics. τ = the 95th percentile of unknownness on *known validation* images, so 95% of knowns are accepted by construction — never tuned on unknowns.

AUROC is threshold-free: the probability a random unknown scores more unknown than a

random known (50% = chance, 100% = perfect). **FPR@95TPR** is the operational number: what fraction of unknowns sneak through when you insist on keeping 95% of knowns. **CSA** = closed-set accuracy, i.e. ordinary CIFAR-10 test accuracy — it checks that rejection was not bought by breaking the classifier.

Table 1 — post-hoc scores on the Vanilla model (AUROC %)

Score	Near	Far	All
MSP	80.96	90.25	85.61
MLS	79.11	90.14	84.63
Energy	79.13	90.25	84.69
Mahalanobis	79.57	92.18	85.88

CSA 94.40. **The near/far gap (~10 points) is the headline of the whole task** — same model, same threshold, and semantic similarity alone costs ten points of separability. Mahalanobis is best on far (92.18) because far unknowns genuinely sit outside the training feature cloud; MSP is best on near because near unknowns land *inside* it and only the head's uncertainty distinguishes them.

Table 2 — trained methods (MLS unless noted)

Model	CSA	Near	Far	All
Vanilla	94.40	79.11	90.14	84.63
GCSC	95.14	80.54	90.85	85.69
PROSER	94.26	79.37	88.90	84.14
PROSER (detection score)	94.26	78.62	87.74	83.18

GCSC's CSA gain over Vanilla is significant ($p_{\text{holm}} = 0.0013$) and its AUROC rises with it — modest support for the "good closed-set classifier" claim. PROSER's dedicated detection score is worse than plain MLS on the same model, which is the kind of negative result worth stating plainly.

Operating point. At $\tau = -5.776$ (Vanilla + MLS), 560 of 800 near unknowns and 361 of 800 far unknowns are still accepted. AUROC of 85 sounds respectable; 70% of near unknowns getting waved through at a usable threshold is what it means in practice. Put both numbers next to each other in the report.

The BatchNorm finding — your best contribution

PROSER's training visibly broke: validation accuracy fell 94.64 $\rightarrow$ 89.46%, the dummy logits won on **100%** of known validation images, and the best checkpoint froze at epoch 1. The cause was not the loss — it was BatchNorm. The mixup blends PROSER feeds through the network are not real images, so the running statistics BatchNorm accumulated during training described blends, not photographs. At evaluation time those wrong statistics shifted every activation.

The diagnosis, run on CIFAR-10 only (no unknowns, so no leakage):

	Before	After BN re-estimation
PROSER val accuracy	89.46%	95.02%
PROSER dummy-wins	100%	2.32%
Vanilla control	94.92%	94.88%

Weights untouched — only the stored means and variances were re-estimated from real training images. The Vanilla control barely moves, which rules out the procedure itself being responsible. The shift concentrates in layer4 (0.062 SD) and layer3 (0.011) versus ≤ 0.003 elsewhere: exactly where the deepest, most semantic features live.

The fixed variant (`proser_bnca1` , run post-hoc with its own later freeze record at 08:56:38 pointing back at the main one): epoch 37, val 95.10%, dummy-wins 2.32%. CSA 94.57; MLS AUROC near 77.95 / far 91.60 / all 84.77; detection score 78.48 / 91.25 / 84.87. The far-unknown gains are significant — +1.46 (MLS) and +1.11 (detection) over Vanilla, +2.69 and +3.51 over the original PROSER. Near and all are not significant, and CSA differences are not significant.

How to present it. Keep the original PROSER as the pre-registered result and the variant clearly labelled post-hoc — that is exactly what your two freeze records establish, and it is more credible than quietly replacing the number. The honest sentence: *PROSER's*

failure was an artefact of BatchNorm statistics estimated on mixup samples; correcting it post-hoc recovers closed-set accuracy in full and yields significant far-unknown AUROC gains, while leaving the near-unknown gap untouched.

The statistics, in plain words

You ran one seed per configuration, so the statistics are doing real work: they are what stops a 1.5-point difference from being written up as a discovery.

Macro-F1. Precision = of the images I called "dog", how many were dogs. Recall = of the actual dogs, how many did I find. F1 = their harmonic mean (harsh on either being low). *Macro* = compute F1 per class, average the classes equally. Use it whenever collapse to a few classes is possible — Task 3's $\lambda=1$ run scored 1.12 macro-F1 on Sketch while predicting one class for everything, which accuracy alone would have flattered.

Confidence interval. A range that would contain the true value most of the time if you repeated the experiment. For accuracies you used binomial / Wilson intervals (Wilson behaves properly near 0% and 100%, where the textbook interval does not). A CI answers "how precise is this number"; it does not by itself answer "are these two numbers different".

McNemar's test. The right test when two models are evaluated on *the same images*. Ignore the images both got right and both got wrong — they carry no information about which model is better. Count only the **discordant pairs**: n_{10} (A right, B wrong) and n_{01} (A wrong, B right). Under the null hypothesis that the models are equally good, each discordant image is a coin flip, so you compute the exact binomial probability of a split at least as lopsided as yours. A small p means the imbalance is hard to explain by chance.

What a p-value is not. It is not the probability the models are the same, and "not significant" is not proof of equality — it means *this experiment cannot tell them apart*. That is the correct phrasing for your Task 3 comparisons.

Holm correction. Run 10 tests at $p < 0.05$ on identical models and you expect about one false positive. Holm fixes this: sort the p-values, compare the smallest to α/n , the next to $\alpha/(n-1)$, and so on, stopping at the first failure. It is uniformly better than Bonferroni and makes no extra assumptions. Applied **within families** — the Task 3 DG comparisons are one family, the Task 4 CSA comparisons another — because correcting across unrelated questions needlessly destroys power. Report `p_holm`, and say which family it was corrected in.

Bootstrap (for AUROC). AUROC is not a count of correct answers, so McNemar does not apply. Instead resample the test set with replacement $B = 2000$ times (stratified: resample knowns and unknowns separately, preserving their proportions), recompute AUROC each time, and read the interval off the spread. Seed 6304, so it reproduces.

Selection noise. Choosing the best epoch on a validation set is itself a choice made on noisy data, so the selected model's validation accuracy is optimistically biased. Worth one sentence in Limitations, especially where best epochs differ wildly across methods (Task 2: epoch 1 for CDAN, 18 for DANN).

The single-seed limitation. State it explicitly: one seed means run-to-run variation is unmeasured, and the significance tests address sampling variation over *images*, not over *training runs*. It is the honest caveat, and stating it yourself is better than having it pointed out.

The cross-task story for your discussion section

The report needs one argument, not four disconnected task summaries. Use the preserve / suppress / reject frame from section 1.

Preserve. Task 1 gives the sharpest version: four models within 1.0 point of each other clean (96.4–97.4) rely on measurably different cues — shape bias 49.6% (ResNet-50) to 85.4% (CLIP zero-shot) — and AdaIN restyling relocates the representation almost entirely (normalised cosine stability 0.04–0.16). So closed-set accuracy does not identify what a model preserves. Tasks 2–3 then show what survives a real distribution shift: ERM keeps 69.59 on Sketch from 94.14 on source, a 25-point gap no method closed.

Suppress. This is where your strongest negative result lives. **Removing domain information is achievable and does not help.** Task 2's DAN drove separability 99.86 → 82.28 and lost 11 points. Task 3's DAN-DG $\lambda=0.1$ drove source separability 87.71 → 79.07 and lost 1.43 (not significant). Push harder and it collapses ($\lambda=1$: separability 48.5, macro-F1 1.12). The λ and α sweeps make this monotone rather than anecdotal: *alignment strength trades against accuracy across the whole range tested, with no beneficial region.*

SAM sharpens the same point from a different angle. It is not an alignment method, it verifiably did what it claims (sharpness +0.286 → +0.110), and it still did not transfer. Two unrelated mechanisms, both confirmed working, neither helping — that is a much stronger statement than either alone.

Reject. Task 4 shows rejection is a separate capability, not a by-product of a good classifier. The near/far gap (~79 vs ~90 AUROC on the same model, same threshold) says difficulty is governed by **semantic distance**, which is the same variable Tasks 2–3 vary as domain distance. GCSC gives the one positive transfer in the assignment: a significantly better closed-set classifier ($p_{\text{holm}} = 0.0013$) was also a slightly better open-set one. Against that, PROSER's purpose-built detection score underperformed plain MLS on the same weights.

The four sentences that could carry the discussion

1. Across two settings and three mechanisms, **measuring that a mechanism worked and measuring that it helped are different measurements** — and here they came apart every time (DAN aligned, SAM flattened, PROSER placed dummies; none transferred).
2. **Distance governs difficulty in both regimes.** Near unknowns cost ~10 AUROC points versus far ones; Sketch costs ~25 accuracy points versus source domains.
3. **Access to unlabelled target data bought nothing measurable** (Task 2 DAN $\lambda=0.1$ vs Task 3 DAN-DG $\lambda=0.1$: -1.15, $p_{\text{holm}} = 0.31$), which questions the premise separating the UDA and DG settings under this budget.
4. **Two failures were artefacts of estimators and statistics, not of the ideas.** The DAN-DG collapse came from the biased MMD estimator's non-zero floor; PROSER's came from BatchNorm statistics estimated on mixup blends. Both were diagnosed and both reversed on correction. Say this — it is the difference between a report that ran code and a report that understood it.

Limitations to state yourself

One seed per configuration; one target domain (Sketch) and one known/unknown pairing (CIFAR-10/100); fixed budgets (≤ 30 epochs, AdamW $1e-4$) that a stronger schedule might change; the plausibility judgement in Task 4 is human and kept deliberately outside the freeze record; Task 1 uses frozen backbones, so its conclusions are about pretrained features, not about what these architectures could learn.

Glossary

Term	Plain meaning
IID	Train and test images come from the same distribution. The assumption this whole assignment breaks.
Inductive bias	A model's built-in preference for one kind of explanation (shape vs texture, say) over another.
Backbone / features / logits	The network body; the number-list it outputs per image; the raw per-class scores before softmax.
Frozen	Weights held fixed; only the linear head trains. Probes what the pretrained features already contain.
ERM	Empirical risk minimisation — plain "minimise average training loss", the no-method baseline.
Domain	A style of image (Photo/Art/Cartoon/Sketch), not a class.
UDA	Unsupervised domain adaptation — unlabelled target images allowed (Task 2).
DG	Domain generalization — no target access at all (Task 3).
MMD	Maximum mean discrepancy: one number for how far apart two clouds of features are.
Biased vs unbiased MMD	Biased keeps self-comparison terms and has a non-zero floor a collapsing model can exploit; unbiased removes them and can go negative.
RBF kernel / median heuristic	The similarity function inside MMD; its width set from the data's own median pairwise distance.
Gradient reversal	A layer that flips the gradient's sign on the backward pass, letting one optimiser train a network and its adversary at once.

Term	Plain meaning
Discriminator	The auxiliary head trying to guess which domain a feature came from.
CDAN	Conditional DANN: the discriminator sees feature $\otimes$ class-probability, so alignment must hold per class.
Domain separability	Accuracy of a logistic-regression probe predicting domain from frozen features. Chance = 50% (2 domains) or 33.3% (3).
SAM	Sharpness-aware minimisation: step uphill by ρ , take the gradient there, apply it back — seeks flat minima.
Sharpness proxy	Loss increase after one normalised gradient-ascent step of radius 0.05. Big = sharp, small = flat.
AdaIN	Adaptive instance normalisation — repaints image A with image B's texture statistics; makes cue-conflict images.
Shape bias / coverage	$N_{\text{shape}} / (N_{\text{shape}} + N_{\text{texture}})$; and the fraction of conflicts where the model chose either source at all.
BatchNorm running statistics	Per-channel means and variances stored during training and used at test time. Wrong ones break a correct model.
OSR	Open-set recognition — classifying knowns while rejecting classes never trained on.
Near / far unknowns	Unknown classes semantically close to a known class (wolf) vs unrelated (wardrobe).
MSP / MLS / Energy	Max softmax probability; max logit; $-\log \text{sumexp}$ of logits. Three ways to read confidence.
Mahalanobis score	Distance to the nearest class centre in feature space under a shared covariance.

Term	Plain meaning
GCSC	"Good closed-set classifier" — the claim that a better closed-set model rejects better for free. Here: RandAugment(2,9).
PROSER	Adds dummy "other" logits trained on manifold-mixup blends to occupy the space between known classes.
Manifold mixup	Blending two images' intermediate features with weight $\lambda \sim \text{Beta}(2,2)$.
τ (threshold)	Unknownness cut-off, set at the 95th percentile on known validation images.
AUROC	Probability a random unknown scores more unknown than a random known. 50% chance, 100% perfect.
FPR@95TPR	Fraction of unknowns accepted when 95% of knowns are kept. The operational number.
CSA	Closed-set accuracy — ordinary test accuracy, checked so rejection is not bought by breaking the classifier.
Freeze record	SHA-256 hashes of the code and outputs, timestamped before unknowns were first touched — evidence against leakage.
Macro-F1	Per-class F1 averaged equally over classes; punishes collapse to a few classes.
McNemar's test	Paired test on the same images; uses only the cases where the two models disagree.
Holm correction	Adjusts p-values for running several tests in one family, without Bonferroni's loss of power.
Stratified bootstrap	Resample knowns and unknowns separately, $B = 2000$ times, to get an interval for AUROC.